

CUDA Special Course Project Report



License Plate Recognition on GPU using CUDA

Dragos-Daniel Bonaparte
s256878@student.dtu.dk

May 2026

Contents

1	Introduction	3
2	Pipeline Overview	3
3	System Architecture	3
4	Pipeline Implementation	4
4.1	Preprocessing	6
4.2	CCL	6
4.3	Plate localisation	7
4.4	Character recognition	7
5	Performance Optimisations	7
5.1	Pre-allocated CCL workspaces	7
5.2	Cooperative propagation kernel	8
5.3	4-slot ping-pong pipeline	8
6	Experimental Setup	9
6.1	Platforms	9
6.2	Dataset and metrics	9
6.3	Compilation and measurement	9
7	Results	10
7.1	Throughput	10
7.2	Recognition accuracy	10
7.3	Max-plates sweep	10
8	Profiling Analysis	11
8.1	CUDA API time	11
8.2	GPU kernel time	12
9	Discussion	12
9.1	The GH200's early bottlenecks	12
9.2	Comparing the two GPUs at this scale	12
9.3	Correctness versus the CPU baseline	13
9.4	Other optimizations that could be explored	14
10	Conclusions	14
A	Build and Run	15
A.1	Build requirements	15
A.2	Runtime inputs	15
A.3	Command-line parameters	15

Abstract

This project ports a single-threaded OpenCV license-plate pipeline to CUDA C++. Every stage becomes a GPU kernel with no OpenCV dependency. The goal of the project is to see how much speedup can be achieved by moving the computation to the GPU while retaining accuracy. The dataset contains 550 images, some which are easy to read and see, and some that are in bad lighting conditions or blurry. The pipeline achieves 932.6 img/s on an RTX 4070 Max-Q and 727.4 img/s on a GH200 — $6.50\times$ and $5.07\times$ over the CPU single-threaded baseline. The surprising findings are that at 1280×720 the workload is clock-bound, not bandwidth- or SM-bound. The image fits in L2 and leaves most of the GH200’s 132 SMs idle, so the smaller, higher-clocked GPU stays $1.29\times$ ahead; the GH200 would most likely win at processing images with much larger resolutions. Against a hand-labelled ground truth, the GPU unfortunately reads plates less accurately than the OpenCV reference.

1 Introduction

This is a license plate recognition pipeline written entirely in CUDA C++. The pipeline is a hand-written port of an existing single-threaded OpenCV reference implementation [7].

The project aimed to parallelize the project in CUDA, not to create a new recognition method: the vision algorithms (adaptive thresholding [2, 3], Otsu’s method [1], union-find connected-component labelling [4, 5], k -NN) are standard and are reimplemented as CUDA kernels. Section 7 reports both *throughput* (the headline result) and *recognition accuracy* against a hand-labelled ground truth, so despite speedup being an important metric, it is not the only consideration. The key finding (Section 9) is that at 1280×720 this workload is clock-bound: the smaller RTX 4070 outperforms the datacenter GH200, whose extra SMs and bandwidth are underutilized at these image resolutions.

All platforms process the same 550 images (1280×720 , .ppm) in the same sorted order; the experimental setup is detailed in Section 6.

2 Pipeline Overview

The pipeline is built from the following stages, every one a hand-written CUDA kernel validated against the CPU reference:

- Background loader thread plus a 4-slot ping-pong upload pipeline (Section 5.3).
- Scene preprocessing: eight kernels, BGR to a binary threshold image (Section 4.1).
- Connected-component labelling with a cooperative propagation kernel, then blob filtering (Section 4.2).
- Character grouping on the CPU (distance, angle, height similarity; Section 4.3).
- Batched warp-crop: all plate candidates rectified in one kernel launch (Section 4.3).
- Per-plate recognition: preprocess, resize, Otsu threshold, CCL, then $k=1$ KNN (Section 4.4, Listings 1 and 2).
- Pre-allocated GPU and pinned-host memory; no `cudaMalloc` in the scene loop (Section 5.1).
- End-to-end benchmarking with CSV output (Section 7).

3 System Architecture

Figure 1 shows the module structure. `PipelineContext` owns all GPU and pinned-host memory. It allocates four `SceneBuffer` slots and one `PlateBuffer` per plate slot. No `cudaMalloc` or `cudaFree` runs during the processing loop.

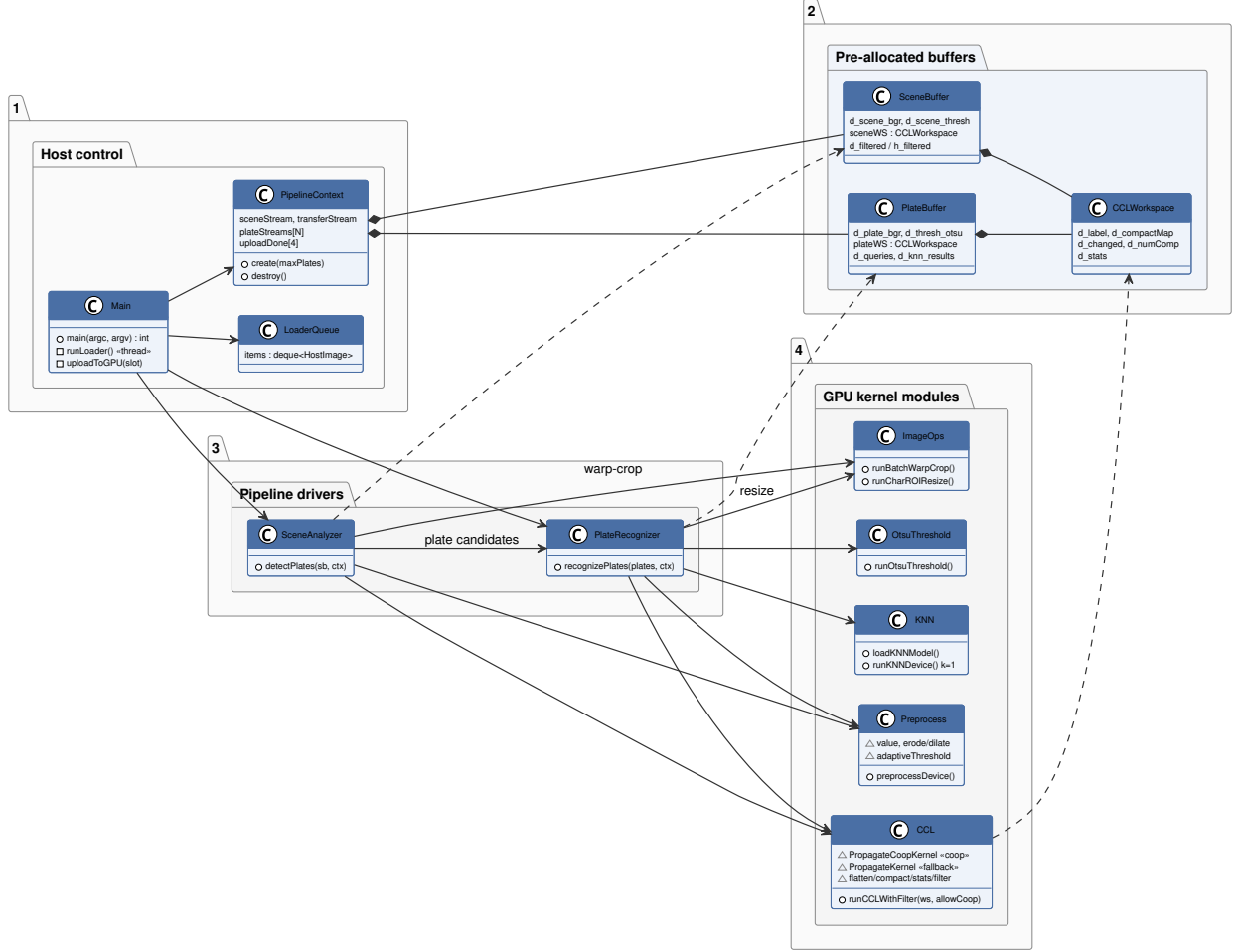


Figure 1: Module architecture, grouped in four layers: host control, pre-allocated buffers, pipeline drivers, and GPU kernel modules. Diamonds denote composition; the bold arrow is the scene-to-plate handoff. Each **CCLWorkspace** is pre-allocated inside its **SceneBuffer** or **PlateBuffer**.

Three CUDA streams are used:

- **sceneStream** – scene kernels (preprocess, CCL, warp-crop).
- **transferStream** – async H→D uploads, decoupled from compute.
- **plateStreams[N]** – one per plate slot; Wave 1 (CCL) and Wave 2 (KNN) run all N in parallel.

The scene stream is gated by four `uploadDone` events, one per scene slot (`NUM_SCENE_SLOTS = 4`, the ping-pong depth of Section 5.3). Each event is recorded on `transferStream` once a slot’s async upload completes; `sceneStream` then blocks on it with `cudaStreamWaitEvent` before reading that slot, so a kernel never touches a frame whose H→D copy is still in flight. This event handshake is the synchronisation half of the ping-pong pipeline detailed in Section 5.3.

4 Pipeline Implementation

Figure 2 traces one image through the pipeline.

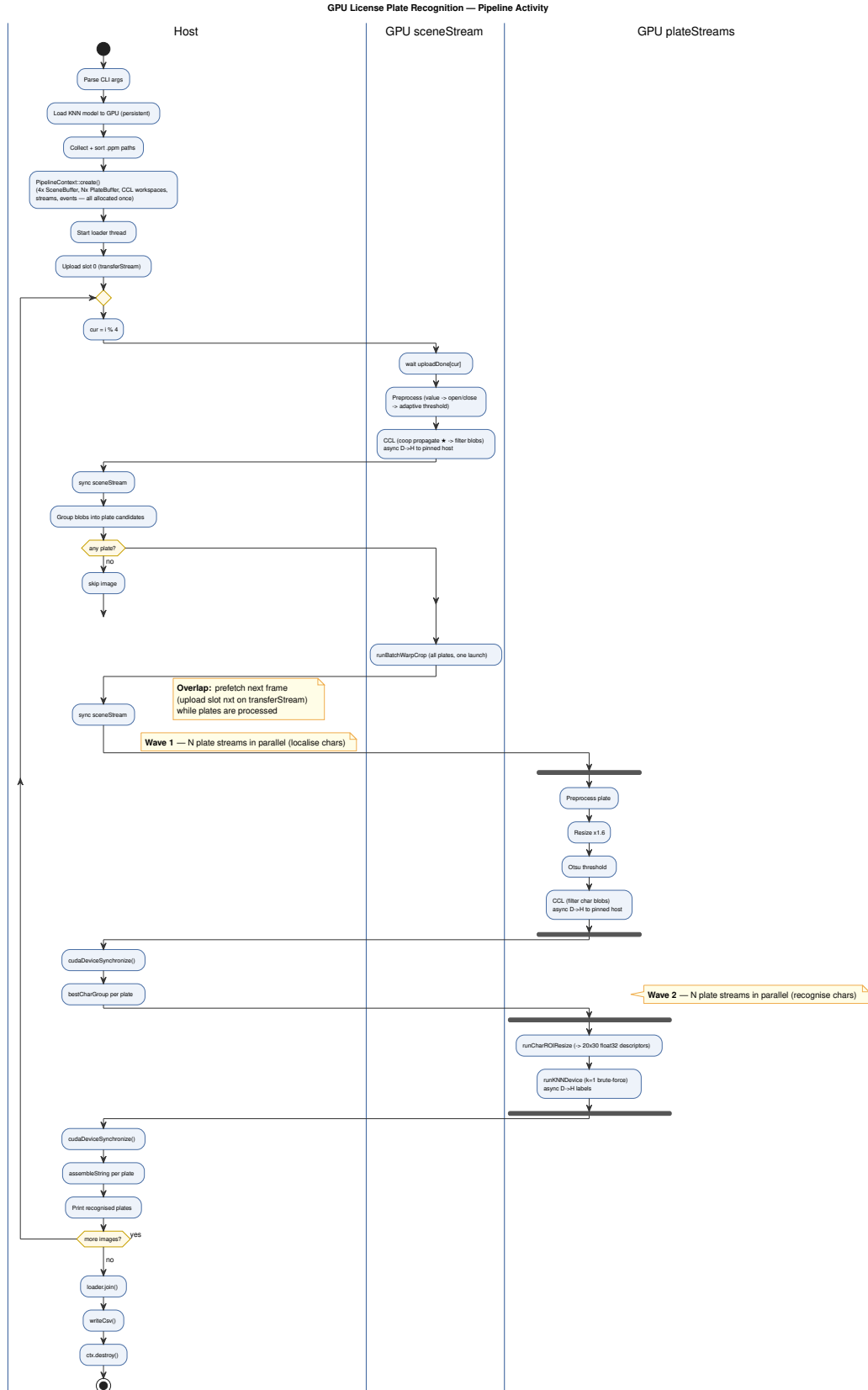


Figure 2: End-to-end pipeline. The Host lane drives CPU logic and sync points. The scene stream runs as one vertical pass. Wave 1 and Wave 2 each launch N plate streams in parallel, separated by a device sync.

The pipeline runs in two stages. The *scene stage* turns the full frame into character-sized blobs: preprocess, connected-component labelling (CCL), then group blobs into plate candidates. The *plate stage* processes each candidate on its own: rectify, threshold, a second CCL to split out characters, then KNN to read them.

4.1 Preprocessing

Eight kernels run in sequence on the scene stream, mirroring the OpenCV reference’s preprocessing. They all operate on the *value channel* — the brightness (V) channel of the frame, where the bright/dark contrast of plate characters is strongest:

1. **Value extraction** — pull the brightness channel out of the BGR frame.
2. **Opening** (erode then dilate, two kernels) — *erode* shrinks bright regions and *dilate* grows them back; doing erode-then-dilate removes small bright specks (noise) while leaving larger shapes intact.
3. **Closing** (dilate then erode, two kernels) — the opposite order, which fills in small dark gaps inside bright regions.
4. **Contrast maximisation** — combine the opened and closed images to widen the gap between light and dark, so characters stand out from the background.
5. **Gaussian blur** — lightly smooth the result to suppress leftover pixel noise before thresholding.
6. **Adaptive threshold** — the final and heaviest kernel, which converts the grey image into a pure black-and-white (binary) image, `d_scene_thresh`.

The adaptive threshold decides black-or-white per pixel by comparing each pixel to a Gaussian-weighted average of its *local* neighbourhood, rather than to one global cut-off. Using a local average means a bright patch of road and a shadowed patch are each judged against their own surroundings, so uneven lighting across the scene does not wash out the characters.

4.2 CCL

Six kernels run in order:

1. Init – each foreground pixel gets its own label.
2. Propagate – iterative union-find with `atomicMin`; convergence is checked via `grid.sync()` in the cooperative variant (Section 5.2).
3. Flatten – path-compress all labels to their roots.
4. Compact – assign dense component IDs via `atomicAdd`.
5. Stats – per-component bounding box and pixel count.
6. Filter – keep blobs within character bounds ($\text{area} > 80$, $w > 2$, $h \geq 8$, $0.25 \leq w/h \leq 1.0$).

The blobs that remain are copied asynchronously to pinned host memory on the same stream.

The core is the propagate kernel: every foreground pixel unions its label with its four neighbours, keeping the smallest root. `atomicMin` makes the merge race-free, and `d_changed` flags that another pass is needed.

4.3 Plate localisation

After the scene sync, `blobsToChars` and `groupChars` cluster character blobs by distance, angle, and height. `runBatchWarpCrop` then crops and rectifies all plate candidates in one kernel launch.

4.4 Character recognition

Each plate crop is preprocessed, resized $\times 1.6$, and Otsu-thresholded. Otsu picks a per-plate threshold from the intensity histogram, adapting to each plate's own contrast (Listing 1).

Listing 1: Otsu threshold: GPU histogram, CPU threshold search, GPU apply (`src/device/OtsuThreshold.cu`).

```
1 # GPU: count how many pixels have each brightness 0..255 (a histogram)
2 each thread adds its pixels into a shared 256-bin histogram (atomicAdd),
3 then adds that block's histogram into the global one (atomicAdd)
4
5 # CPU: try every brightness cut-off, keep the one that splits best
6 for cutoff in 0..255:
7     fracDark    = fraction of pixels darker than cutoff
8     fracLight   = fraction of pixels brighter than cutoff
9     meanDark    = average brightness of the dark pixels
10    meanLight   = average brightness of the light pixels
11    score[cutoff] = fracDark * fracLight * (meanDark - meanLight)^2
12 bestCutoff = the cutoff with the highest score    # groups most separated
13
14 # GPU: pixel >= bestCutoff ? white (255) : black (0)
```

CCL then extracts the characters. Each is resized to a 20×30 descriptor (600 float32 features) and classified by KNN (Listing 2): one block per character, each thread owns one of the 180 training samples, and a shared-memory arg-min reduction picks the nearest.

Listing 2: KNN classify: one block per character, $k=1$ (`src/device/KNN.cu`).

```
1 each block = one query character; each thread = one of 180 samples
2 load the 600-feature query into shared memory; syncthreads()
3 dist[tid] = sum over 600 features of (query[i] - sample[tid][i])^2    # squared L2
4 syncthreads()
5 parallel arg-min reduction over dist[] -> nearest sample
6 thread 0: result = label[nearest]                                     # k = 1
```

5 Performance Optimisations

Before optimizations the RTX 4070 reached ≈ 700 img/s and the GH200 reached ≈ 620 img/s (both at `--max-plates 8`). Three changes have raised those to 932.6 and 727.4 img/s.

5.1 Pre-allocated CCL workspaces

The original `runCCLWithFilter` ran five `cudaMalloc` calls on every frame and every plate crop, then freed them. This was deliberate: `allocWorkspace(width  $\times$  height)` sized the workspace to each image exactly, so the program accepted any image size without knowing the dimensions in advance. At 700 img/s with eight plate slots that is $700 \times 9 \times 5 \approx 31\,500$ allocator calls per second.

Each `SceneBuffer` and `PlateBuffer` now owns a persistent `CCLWorkspace` (five device buffers, allocated once at startup). A `cudaMemsetAsync` resets the counters between frames. The trade-off is the lost flexibility: the workspaces are sized once to fixed maxima using a python script,

(SCENE_W×SCENE_H for scenes, MAX_PLATE_THRESH_W×MAX_PLATE_THRESH_H for plates), which is why the input is now fixed at 1280×720.

	Before	After
cudaMalloc/cudaFree per image	≈90	0
CCL workspace lifetime	per-call	persistent

Table 1: CCL workspace allocation before and after.

5.2 Cooperative propagation kernel

The original convergence loop did one host round-trip per iteration: `cudaStreamSynchronize`, then two blocking `cudaMemcpy` calls (reset the flag H→D, read it back D→H). Each round-trip costs ≈30 μs; 20–50 iterations per scene added 0.6–1.5 ms per image. On the GH200 the kernels are faster, so the fixed round-trip cost is a larger share of total time, which is part of why the GH200 was initially slower (Section 9).

The replacement, `cclPropagateCoopKernel`, is launched with `cudaLaunchCooperativeKernel`. All thread blocks are resident at once, which enables `grid.sync()` inside the kernel which is a barrier that waits for *every* thread in *every* block across the whole grid, not just the threads in one block. Three grid barriers per iteration replace the host round-trips:

Listing 3: Three-sync convergence pattern.

```

1 for (int iter = 0; iter < maxIter; iter++) {
2     if (tid == 0) *d_changed = 0;
3     grid.sync();                      // sync 1: all threads see reset
4
5     /* propagation: atomicMin + atomicOr(d_changed, 1) */
6
7     grid.sync();                      // sync 2: all writes visible
8
9     bool converged = (*d_changed == 0); // cache before sync 3
10
11    grid.sync();                      // sync 3: prevent race on next reset
12    if (converged) break;
13 }
```

Sync 3 is required. Without it the fastest block races ahead and resets `d_changed` before slower blocks have read it, so they break early and deadlock at the next `grid.sync()`.

5.3 4-slot ping-pong pipeline

A background loader thread feeds images into a bounded queue (capacity 3). Uploads run on `transferStream`; a `cudaStreamWaitEvent` on `sceneStream` delays kernel launch until the current slot’s upload finishes. This hides the H→D transfer behind compute across four scene slots.

Figure 3 shows the steady state. Each frame (one colour) flows diagonally down the four lanes: load, then upload, then scene compute, then plate compute. At any instant four different frames occupy the four lanes, so the H→D upload of one frame runs while the GPU computes another.

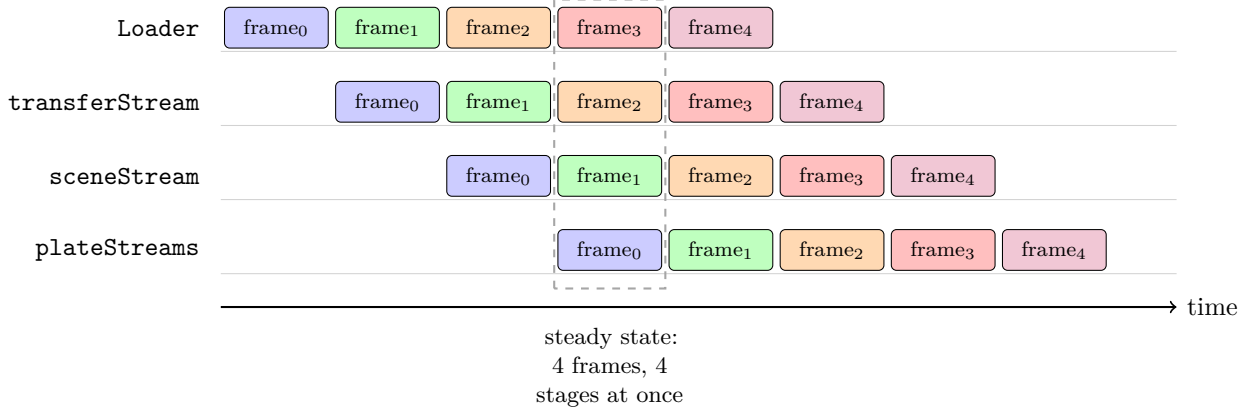


Figure 3: 4-slot ping-pong in steady state. Time runs left to right; lanes are stacked in dependency order. Each frame (one colour) steps diagonally: load $\rightarrow$ upload $\rightarrow$ scene $\rightarrow$ plate. The dashed column shows that at one instant frame₃ loads, frame₂ uploads, frame₁ runs scene kernels, and frame₀ runs plate kernels, so transfer is fully hidden behind compute.

6 Experimental Setup

6.1 Platforms

Platform	GPU	VRAM	Notes
CPU baseline	—	—	AMD Ryzen 7 7745HX, single-threaded OpenCV
NVIDIA RTX 4070 Max-Q	Ada Lovelace	8 GB GDDR6	36 SMs, 3105 MHz max, PCIe Gen4
NVIDIA GH200	Hopper (H100)	96 GB HBM3e	132 SMs, 1980 MHz max, NVLink-C2C

Table 2: Evaluation platforms.

The CPU baseline runs single-threaded on an AMD Ryzen 7 7745HX (8 cores, 16 threads, up to 5.15 GHz boost, 32 MB L3).

6.2 Dataset and metrics

All runs use the same 550 images, some of which are easy to read and see, and some that are in bad lighting conditions or blurry. The CPU reads the images in the `.jpg` format, while the GPU reads them in the `.ppm` format. The frames are processed in the same order, so frame i is identical across platforms. Wall-clock time covers the full loop, from the first upload to the final `cudaDeviceSynchronize`. The primary metric is throughput (img/s); recognition quality is reported separately in Section 7.2 against a hand-labelled ground truth of 119 frames.

6.3 Compilation and measurement

The binary is compiled with `-O3 -arch=sm_89` for the RTX 4070 and `-arch=sm_90` for the GH200; cooperative launch requires `-rdc=true`. Each configuration was run five times back-to-back and the steady-state run reported. The first 2 runs are discarded and considered warm-up.

7 Results

7.1 Throughput

Platform	Throughput (img/s)	Speedup vs CPU
CPU baseline	143.4	1.00×
RTX 4070 Max-Q	932.6	6.50×
GH200	727.4	5.07×

Table 3: Final benchmark results on the 550-image set (`--max-plates 8`). Speedups are relative to the single-threaded CPU baseline (Section 6).

The CUDA pipeline reaches 6.50× the single-threaded OpenCV baseline on the RTX 4070 and 5.07× on the GH200. Section 9 will explain the assumptions of why the RTX4070 is faster than the GH200.

7.2 Recognition accuracy

Throughput only matters if the pipeline still reads the right plates, so 119 frames spanning the sequence were hand-labelled to form a ground truth. For each labelled frame a pipeline is scored on its best printed candidate. Three quantities are reported (Table 4): the *detection rate* (fraction of ground-truth frames for which at least one plate string was emitted), the *exact-match rate* (best candidate equals the label character-for-character), and *character similarity*

Metric (over 119 ground-truth frames)	CPU (OpenCV)	GPU (CUDA)
Detection rate (%)	68.9	51.3
Exact plate match (%)	7.6	2.5
Character similarity (%)	33.6	23.9

Table 4: Recognition accuracy against hand-labelled ground truth.

Two things stand out. First, accuracy is low on both pipelines. Perhaps the dataset chosen as benchmark is too difficult for both algorithms given that even the OpenCV reference rarely gets a plate exactly right. Second, the GPU is clearly doing worse than the OpenCV baseline. It finds a plate in 51.3% of labelled frames, while the CPU finds one in 68.9%, and it matches the full plate exactly far less often. The handwritten CUDA detection, thresholding, and brute-force KNN are simpler than OpenCV’s versions, and they lose accuracy on the hardest frames. When both pipelines do return a plate, they agree exactly only 5.0% of the time and reach 51.4% character similarity. Section 9.3 looks at a run of consecutive frames one by one.

7.3 Max-plates sweep

`--max-plates` sets the number of pre-allocated plate slots and parallel plate streams: a low value runs fewer plate streams (higher throughput) but caps how many plates per frame can be recognised (lower recall). Table 5 sweeps it on both GPUs. `--max-plates 1` is the fastest setting (1060 img/s on the RTX 4070) but only finds 423 plates across the set; `--max-plates 8` gives up ≈12% throughput to find 927 plates, and detection does not increase much afterwards. The headline results (Table 3) use `--max-plates 8` because the program is meant ultimately to detect license plates, not necessarily peak throughput, and is what makes the accuracy in Table 4 meaningful.

max-plates	RTX 4070 Max-Q		GH200	
	img/s	Plates	img/s	Plates
1	1060.2	423	813.2	423
2	991.4	658	769.9	654
4	948.6	863	734.2	849
8	932.6	927	727.4	903
16	934.8	931	725.0	903
32	928.6	928	722.4	899
RTX/GH200 ratio		$\approx 1.29\times$ (constant)		

Table 5: Throughput sweep over `--max-plates`, both platforms.

- The RTX/GH200 ratio is $\approx 1.29\times$ at every value. This is a structural gap, not load-dependent.
- Throughput drops from $N=1$ to $N=4$ (-10%), then flattens (-0.9% from 4 to 8). Scene work (adaptive threshold + CCL) dominates; plate streams add little once the scene is done.
- Detection saturates at $N=8$: $N=8$ and $N=16$ find the same number of plates, so $N=8$ is the best operating point.
- The GH200 degrades more gracefully (-10.5% from 1 to 8 vs -12.0% on the RTX 4070) because its 132 SMs absorb parallel plate streams better.

8 Profiling Analysis

Both GPUs were profiled with `nsys profile --trace=cuda,nvtx` over the same 550 images.

8.1 CUDA API time

API call	RTX 4070		GH200	
	%	Calls	%	Calls
<code>cudaStreamSynchronize</code>	39.5	2 948	59.9	2 904
<code>cudaMemcpyAsync</code>	21.0	5 546	6.3	5 458
<code>cudaMalloc</code>	12.9	1 158	13.7	1 136
<code>cudaLaunchKernel</code>	9.9	28 207	6.9	27 728
<code>cudaMemcpy (blocking)</code>	5.1	5 504	7.1	5 382
<code>cudaDeviceSynchronize</code>	2.3	850	3.4	838
<code>cudaLaunchCooperativeKernel</code>	0.2	550	0.2	550

Table 6: CUDA API summary, both platforms (550 images).

`cudaStreamSynchronize` is the most time-consuming API call on both platforms, but more so on the GH200 (59.9% vs 39.5%). It comes from two explicit syncs per image in `SceneAnalyzer` plus one per plate in the plate CCL loop. The GH200 pays a larger share because its kernels finish faster while its host round-trips do not, so the fixed sync cost is a bigger fraction of total time (Section 9). `cudaMalloc` costs $\approx 13\%$ on both (≈ 1150 calls) from kernels which still allocate per call. The RTX spends more of its API time in memory copies (PCIe); the GH200 spends more in stream syncs. `cudaLaunchCooperativeKernel` costs only 0.2% on both.

8.2 GPU kernel time

Kernel	RTX 4070 %	GH200 %
<code>adaptiveThresholdKernel</code>	26.5	59.4
<code>cclPropagateCoopKernel</code>	21.7	14.9
<code>cclStatsKernel</code>	13.1	7.8
<code>erodeKernel</code> / <code>dilateKernel</code>	13.4	3.8
<code>gaussianBlurKernel</code>	5.0	1.2
<code>extractValueKernel</code>	4.3	1.4
<code>cclPropagateKernel</code>	3.5	3.1
<code>knnKernel</code>	3.5	3.0
Other	9.0	5.4

Table 7: GPU kernel time share, both platforms (550 images).

`adaptiveThresholdKernel` is the most expensive kernel on both, and far more expensive on the GH200 (59.4% vs 26.5%). `cclPropagateCoopKernel` runs once per image (550 times) and is the scene CCL. `cclPropagateKernel` is plate CCL (≈ 2700 instances, ≈ 5 iterations/image): fast kernels, but bound by the host round-trip per iteration. `knnKernel` costs $\approx 3\%$ on both; brute-force $k=1$ over 180 samples should not be a bottleneck.

9 Discussion

9.1 The GH200’s early bottlenecks

The GH200 first reached only ≈ 620 img/s versus ≈ 700 img/s on the RTX 4070, despite $3.7\times$ more SMs and $13\times$ more memory bandwidth. Two bottlenecks that do not scale with SM count explain this:

1. Allocator contention. The CUDA allocator is a global lock. The GH200’s faster kernels drive a higher frame-rate attempt, so they hit the allocator more often per second and serialise harder.
2. Host round-trip penalty. The CCL fallback loop costs $\approx 30\mu\text{s}$ per iteration regardless of SM count. The GH200 finishes each kernel iteration faster, so the fixed overhead is a larger fraction of per-frame time.

Pre-allocated workspaces remove the allocator pressure. The cooperative kernel removes the round-trips for scene CCL, which is the dominant CCL call.

9.2 Comparing the two GPUs at this scale

After the fixes the RTX 4070 leads by $\approx 1.29\times$ at every configuration. The raw SM clock ratio is $3105/1980 = 1.57\times$ (from `nvidia-smi`). The observed gap ($1.29\times$) is narrower than the clock gap ($1.57\times$): the GH200 architecture recovers $\approx 18\%$, but cannot close it at this image scale.

The GH200’s advantages that do not help here:

1. NVLink-C2C bandwidth ($28\times$ faster H $\rightarrow$ D). The ping-pong pipeline already hides transfers. A 2.76 MB frame takes $206\mu\text{s}$ on PCIe and $7\mu\text{s}$ on NVLink-C2C; both are shorter than the ≈ 1 ms compute time, so neither platform is transfer-bound.

2. HBM3e bandwidth ($13\times$ higher). The 0.9 MB scene fits in both GPUs’ L2 caches (RTX 4070: 36 MB, GH200: 50 MB). No DRAM transaction occurs in steady state, so the bandwidth gap is invisible.
3. SM count ($3.7\times$ more). At 1280×720 the cooperative CCL launches $132 \times 2 = 264$ blocks for 921 600 pixels: about 13 pixels per thread. Most GH200 SMs sit idle; this workload is per-SM throughput, where clock speed wins.

Table 8 shows measured bandwidth and latency from `nvbandwidth v0.9`.

Metric	GH200	RTX 4070	Ratio
H2D bandwidth, CE (GB/s)	380.1	13.4	$28.4\times$ GH200
D2H bandwidth, CE (GB/s)	297.7	13.5	$22.1\times$ GH200
Bidirectional H2D, CE (GB/s)	178.3	13.3	$13.4\times$ GH200
Bidirectional D2H, CE (GB/s)	189.0	11.6	$16.3\times$ GH200
CPU–GPU latency, SM (ns)	617.7	563.7	$1.09\times$ RTX faster
Device local copy (GB/s)	1462.4	112.7	$13.0\times$ GH200

Table 8: Bandwidth and latency from `nvbandwidth v0.9` (GH200 NVLink-C2C vs RTX 4070 PCIe).

The GH200’s CPU–GPU latency (617.7 ns) is 9.5% higher than the RTX 4070 (563.7 ns). The plate CCL fallback issues two latency-bound 4-byte `cudaMemcpy` calls per iteration; across 2690 iterations that adds ≈ 0.3 ms. NVLink-C2C does not reduce small-transfer latency below PCIe in these experiments.

The GH200’s advantages only show with larger workloads. At 4K the CCL and threshold kernels would launch $\approx 9\times$ more blocks, the cooperative kernel could saturates all 132 SMs, and the $13\times$ bandwidth advantage would be obvious.

9.3 Correctness versus the CPU baseline

The aggregate accuracy in Section 7.2 is best understood frame by frame. Table 9 shows a consecutive sample (frames 453–468), a clearer close-up segment of the set where both pipelines do better than their averages, with the hand-labelled ground truth alongside.

Frame	Ground truth	CPU (OpenCV)	GPU (CUDA)	Note
453	MCLRNF1	MCLRNF1	N0LPNE1	CPU exact
454	ZOOMN65	Z00MN65	—	CPU O→0; GPU miss
455	HOR5SH1T	HUR5SH1T	HDH5SH1T	close
456	FALLYOU	FALLY9U	EALLYBB	both poor
457	EZEY	EZEY	EZEY	all exact
458	NITESKY	NITESIY	N1TESKY	close
459	U8NTBAD	U8NTBAD	68NTBAB	CPU exact
460	GAY247	0AY247	8AY247	1 char each
461	LOLWATT	L0LWATT	WATT4	CPU close; GPU partial
462	RIPLS1	RIPLS1	PJPLS1	CPU exact
463	NICESKY	1N1CESY	NT0ESKY	both poor
464	NVSBLE	NVSBLE	—	CPU exact; GPU miss
465	NYSJ	NYSJ	NYSJ	all exact
466	ANBYOND	ANBYOND	ZNBYOYB	CPU exact
467	1ZM961	1ZP961	1ZM961	GPU exact
468	PNYEXPS	PNYEXPS	PNYEIPS	CPU exact

Table 9: Per-frame plate readings, ground truth vs CPU vs GPU (frames 453–468). Only the candidate closest to the ground truth is shown

The two pipelines localise and read the same plates on the same frames: several exact matches, and several more within a single character. Despite rewriting the OpenCV stages by hand in CUDA, the behaviour is not exactly equivalent in every case, and the aggregate in Table 4 confirms the GPU is worse at detection.

9.4 Other optimizations that could be explored

The profiling points to a few areas worth investigating further. The adaptive-threshold kernel is the largest single cost (27–59% of GPU time), so it is the first target for further kernel-level optimisation. On the API side, `cudaMalloc` still costs $\approx 13\%$ (the plate warp-crop and char-resize paths allocate per call) and `cudaStreamSynchronize` takes the largest share (mostly the plate-CCL fallback loop and the per-plate Otsu histogram round-trip, Section 4.4), so both the remaining hot-path allocations and the per-iteration host syncs are clear next steps. The high `cudaLaunchKernel` count ($\approx 28,000$) and the repeated morphology passes also leave launch overhead on the table; the pipeline was split into many small, single-purpose kernels for readability, and fusing them would reduce it. On the recognition side, the modest accuracy in Table 4 suggests the brute-force $k=1$ KNN and the simple blob filter are the limiting factors.

10 Conclusions

The project ported a single-threaded OpenCV license-plate pipeline into a full CUDA implementation with no vision library, reaching 932.6img/s on an RTX 4070 ($6.50\times$ the single-threaded baseline) and 727.4img/s on a GH200 ($5.07\times$). Pre-allocated workspaces and a cooperative-group CCL kernel removed the allocator contention and host round-trips that had initially held the GH200 below the smaller GPU.

The central finding is a scaling one: at 1280×720 this workload is clock-bound, not bandwidth- or SM-bound. The image is small enough to live in L2 on both GPUs and to leave most of the GH200’s 132 SMs idle, so the datacenter card’s bandwidth and core-count advantages never engage and the higher-clocked consumer RTX 4070 wins by a constant $\approx 1.29\times$. The GH200 would only

pull ahead at much larger frames (4K and up), where the kernels launch enough blocks to fill its SMs and the data no longer fits in cache.

The honest counterpoint is recognition quality: against a hand-labelled ground truth the GPU detects fewer plates and matches exactly less often than the OpenCV reference (Section 7.2). Despite that, the project achieved its primary goal of building a full CUDA vision pipeline, and the accuracy gap is a separate issue that could be improved with better algorithms and more engineering effort.

A Build and Run

A.1 Build requirements

- CMake ≥ 3.18 and a CUDA Toolkit (tested with CUDA 12).
- A C++17 host compiler.
- Separable compilation (`-rdc=true`). The cooperative CCL kernel needs it.
- Target architecture `sm_89` (RTX 4070) or `sm_90` (GH200). CMake uses `native` by default.

Listing 4: Build.

```
1 mkdir build && cd build
2 cmake .. && make
```

A.2 Runtime inputs

- Input images must be `.ppm` at 1280×720 . Other sizes are not handled (Section 5.1).
- `knn_data.bin` (the trained KNN model) must sit next to the binary. CMake copies it there at build time.

A.3 Command-line parameters

Parameter	Default	Meaning
<code>[image.ppm dir]</code>	<code>scan cwd</code>	Single image or a directory. If omitted, scans the current (then parent) directory for files named <code>image*.ppm</code> .
<code>--max-plates N</code>	8	Number of pre-allocated plate slots and parallel plate streams. Minimum 1.
<code>[out.csv]</code>	<code>auto</code>	Aggregate benchmark output file. Defaults to <code>license_plate_gpu_benchmark.csv</code> .

Table 10: Program parameters (positional order is flexible; arguments are matched by shape).

Output is two-fold: recognised plate strings are printed to stdout, and one summary row (throughput, plate and character counts) is written to the CSV.

References

- [1] N. Otsu, “A Threshold Selection Method from Gray-Level Histograms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979.

- [2] D. Bradley and G. Roth, “Adaptive Thresholding Using the Integral Image,” *Journal of Graphics Tools*, vol. 12, no. 2, pp. 13–21, 2007.
- [3] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.
- [4] O. Stava and B. Beneš, “Connected Component Labeling in CUDA,” in *GPU Computing Gems Emerald Edition*, W.-m. Hwu, Ed. Morgan Kaufmann, 2011, pp. 569–581.
- [5] Y. Komura, “GPU-based cluster-labeling algorithm without the use of conventional iteration: application to the Swendsen–Wang multi-cluster spin-flip algorithm,” *Computer Physics Communications*, vol. 194, pp. 54–58, 2015.
- [6] NVIDIA Corporation, *CUDA C++ Programming Guide — Cooperative Groups*, 2024. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- [7] C. Dahms, *OpenCV_3_License_Plate_Recognition* (open-source ANPR reference implementation). https://github.com/MicrocontrollersAndMore/OpenCV_3_License_Plate_Recognition_Cpp